

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ИНДИВИДУАЛЬНОЕ ДОМАШНЕЕ ЗАДАНИЕ
по дисциплине «Введение в нереляционные системы управления базами
данных»

Тема: Инструмент для кластеризации логов Apache

Студент гр. 3344

Волков А.А.

Студент гр. 3344

Коршунов П.И.

Студент гр. 3344

Хангулян С.К.

Преподаватель

Заславский М.М.

Санкт-Петербург

2026

АННОТАЦИЯ

В рамках данного курса предполагалось разработать приложение в команде на одну из предложенных тем, связанных с использованием нереляционных СУБД. Была выбрана тема создания инструмента для хранения, поиска, группировки и кластеризации логов Apache с использованием MongoDB, так как такая задача хорошо демонстрирует работу с полуструктурированными данными и аналитическими запросами.

В работе рассматриваются хранение логов, массовый импорт и экспорт данных, фильтрация по нескольким полям, запуск кластеризации и отображение статистики по выбранному подмножеству данных. Отдельное внимание уделяется сравнению нереляционной модели MongoDB с аналогичной реляционной моделью по объему хранения, количеству запросов и удобству реализации сценариев использования.

Исходный код, wiki-страницы с макетом, сценариями использования и моделью данных доступны в репозитории: <https://github.com/moevm/nsql1h26-apache>

ANNOTATION

As part of this course, a team project based on one of the proposed topics related to NoSQL database systems had to be developed. The chosen topic was an Apache log storage, search, grouping and clustering tool based on MongoDB, since this task demonstrates working with semi-structured data and analytical queries.

The project covers storing Apache access/error logs, bulk import and export, multi-criteria filtering, clustering runs and displaying statistics for a selected subset of data. Special attention is paid to comparing the MongoDB document model with an equivalent relational model in terms of storage size, number of queries and convenience of implementing the main use cases.

The source code, UI mockup, use cases and data model are available in the repository: <https://github.com/moevm/nsql1h26-apache>

СОДЕРЖАНИЕ

1 ВВЕДЕНИЕ.....	6
1.1 Актуальность решаемой проблемы.....	6
1.2 Постановка задачи	7
1.3 Предлагаемое решение	7
1.4 Качественные требования к решению	8
2 СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ.....	8
2.1 Макет UI.....	8
2.2 Сценарии использования.....	9
3 МОДЕЛЬ ДАННЫХ	11
3.1 Нереляционная модель MongoDB.....	11
3.2 Реляционная модель	15
3.3 Сравнение моделей	17
4 РАЗРАБОТАННОЕ ПРИЛОЖЕНИЕ	21
4.1 Архитектура и контейнеры	21
4.2 Модули приложения	21
4.3 REST API.....	22
4.4 Методы кластеризации.....	22
4.5 Снимки экрана приложения.....	23
5 ВЫВОД	27
6 ПРИЛОЖЕНИЯ.....	27
6.1 Репозиторий проекта	27

6.2 Документация по сборке и развертыванию приложения	28
6.3 Инструкция пользователя	28
7 СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	29

1 ВВЕДЕНИЕ

Целью индивидуального домашнего задания является разработка приложения для анализа логов Apache с применением нереляционной СУБД MongoDB. В рамках работы требуется показать, что выбранная NoSQL-модель действительно подходит для хранения полуструктурированных записей, поиска по ним и выполнения аналитических операций.

Разрабатываемая система ориентирована на сценарий, когда пользователь получает набор access и error логов, загружает их в приложение, просматривает таблицу записей, задает фильтры и запускает группировку похожих событий. Результаты группировки сохраняются как отдельные запуски.

1.1 Актуальность решаемой проблемы

Веб-серверы постоянно создают логи событий. Эти логи используются для поиска ошибок, анализа нагрузки, выявления подозрительных IP-адресов, исследования популярных endpoint и диагностики нестабильного поведения приложения. При ручном анализе даже несколько тысяч строк становятся неудобными для просмотра, а повторяющиеся ошибки сложно отличить от единичных событий.

Apache access и error логи имеют разную структуру. Access-строка обычно содержит IP, дату, HTTP-метод, URI, статус, размер ответа и User-Agent. Error-строка содержит время, уровень, модуль, процесс, клиента и текст ошибки. Поэтому хранение таких данных в строго реляционной форме приводит либо к широкой таблице с большим количеством пустых полей, либо к сложной схеме с множеством таблиц.

MongoDB позволяет хранить лог как документ: рядом с исходной строкой можно держать поля и нормализованные признаки для анализа. Это снижает сложность чтения, упрощает импорт разных форматов и позволяет переносить фильтрацию, группировку и статистику на уровень СУБД.

1.2 Постановка задачи

Необходимо разработать прототип приложения «Инструмент для кластеризации логов Apache». Приложение должно демонстрировать полный цикл работы с данными: импорт, хранение, представление, анализ, экспорт.

- загрузка Apache логов;
- автоматическое определение типа строки по формату;
- многокритериальная фильтрация по типу, периоду, статусу, методу и подстроке;
- просмотр деталей и исходной строки выбранного лога;
- запуск кластеризации с сохранением истории запусков;
- отображение статистики и диаграмм по выбранному запуску;
- экспорт всех данных приложения в JSON;
- развертывание через Docker Compose.

1.3 Предлагаемое решение

Предлагается реализовать приложение на Python и FastAPI. MongoDB используется как основная СУБД и хранит три смысловые группы данных: исходные логи, запуски кластеризации и рассчитанные кластеры. Приложение использует MongoDB для хранения и агрегации.

При импорте каждая строка файла проверяется на тип access и error. Если строка соответствует одному из форматов, она преобразуется в документ logs. Если строка не распознана, она учитывается как ошибка импорта.

Кластеризация работает как группировка похожих логов по вычисляемому ключу. Для access-логов ключ строится из метода, нормализованного endpoint, статус-кода и IP-префикса. Для error-логов ключ строится из уровня и нормализованного текста сообщения.

1.4 Качественные требования к решению

- Требуется разработать веб-приложение с использованием MongoDB в качестве основной нереляционной СУБД. Приложение должно быть предназначено для хранения, поиска, представления и анализа логов Apache.
- Приложение должно предоставлять пользователю средства массового импорта данных, просмотра сохранённых логов, многокритериальной фильтрации и экспорта данных.
- Аналитическая часть должна позволять выполнять группировку и кластеризацию логов по выбранным признакам, сохранять результаты выполненных запусков и отображать статистику по выбранным данным.
- Разработанное решение должно быть воспроизводимым, переносимым и пригодным для демонстрации в изолированном окружении.

2 СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ

2.1 Макет UI

Макет UI размещен на wiki-странице репозитория. Он отражает минимальный веб-интерфейс: главный экран, импорт, экспорт, таблица логов, запуск кластеризации, просмотр результатов и статистика.

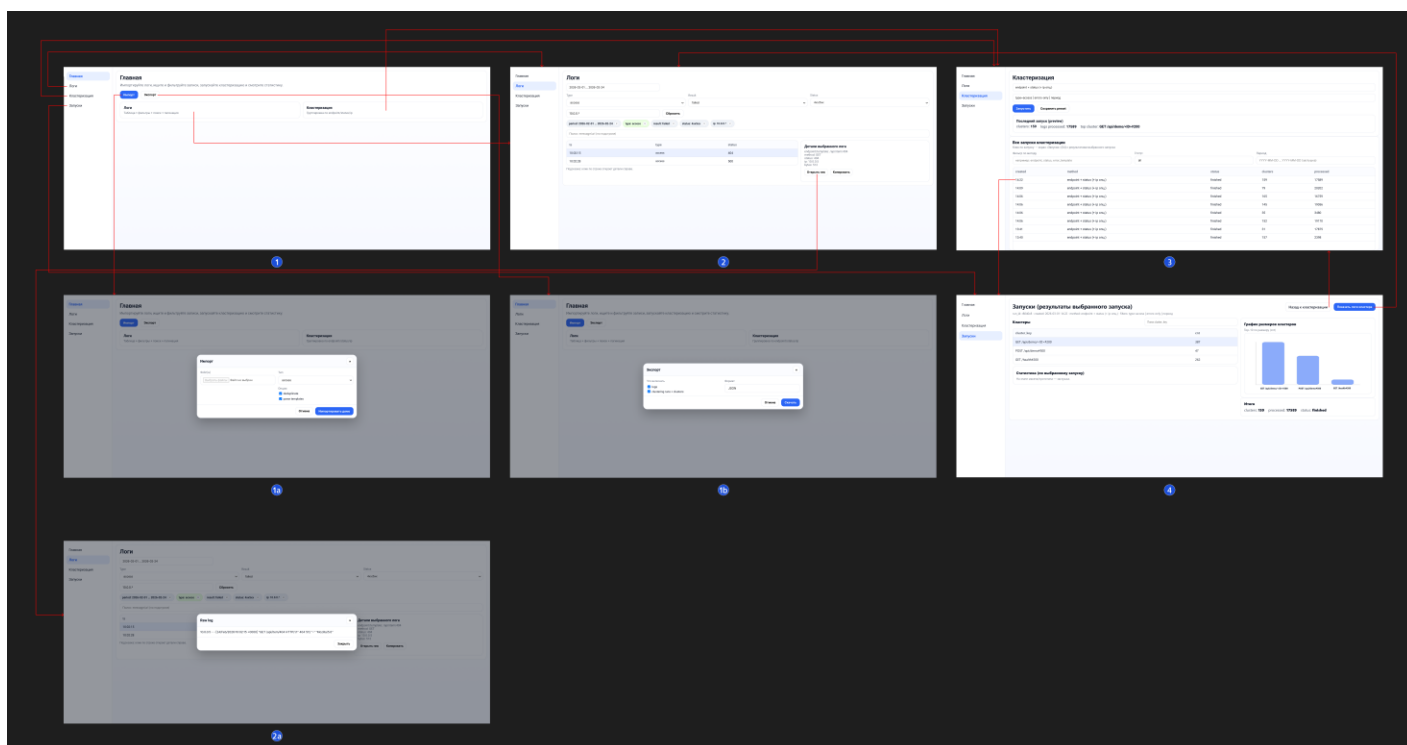


Рисунок 1 - Макет UI

2.2 Сценарии использования

1) Импорт данных

Пользователь открывает раздел импорта, выбирает файл и режим загрузки. Для обычных логов поддерживаются access, error и mixed файлы.

Режим append добавляет новые записи. Режим replace сначала полностью разбирает файл, проверяет наличие валидных логов и только после этого очищает старые записи. Это защищает от ситуации, когда пользователь выбирает неподходящий файл.

2) Представление данных

В разделе логов пользователь видит таблицу с датой, типом, методом, статусом, URI, уровнем ошибки и фрагментом сообщения.

Фильтры включают период с выбором даты/времени, тип лога, статус-код, HTTP-метод и строку поиска. Поиск применяется к raw, URI, сообщению и

другим текстовым полям. В интерфейсе также доступна карточка выбранного лога.

3) Анализ данных

Пользователь выбирает метод кластеризации и фильтры. Например, можно кластеризовать только access-логи со статусами 4xx/5xx за выбранный период. После запуска приложение создает запись `cluster_runs` и сохраняет рассчитанные кластера. Статистика выбранного запуска показывает топ-кластеры, распределение по статусам, типам логов и HTTP-методам.

Кастомизируемая статистика строится по выбранному подмножеству данных. Пользователь задает фильтр и выбирает атрибуты для осей диаграммы: например по оси X статус-код, по оси Y метод. В результате отображается количественная диаграмма распределения.

4) Экспорт данных

Пользователь открывает экран "Главная". Пользователь нажимает кнопку "Экспорт". Экспорт отдает JSON-дамп, содержащий `logs`, `cluster_runs` и `clusters`.

Для разрабатываемого приложения будут преобладать операции чтения данных.

Запись выполняется в основном при массовом импорте логов и при сохранении результатов запуска кластеризации. Эти операции происходят относительно редко: пользователь загружает набор логов, после чего многократно просматривает, фильтрует и анализирует уже сохранённые данные.

Основные пользовательские сценарии связаны с чтением: просмотр таблицы логов, поиск по нескольким полям, просмотр деталей записи, открытие

ранее выполненных запусков кластеризации, просмотр кластеров и построение статистики.

3 МОДЕЛЬ ДАННЫХ

3.1 Нереляционная модель MongoDB

Основная идея NoSQL-модели состоит в том, что один лог хранится как один документ. Это естественно для Apache-логов, потому что разные типы строк имеют разный набор полей, а набор распарсенных атрибутов может расширяться без миграции схемы таблицы.

Нереляционная модель данных

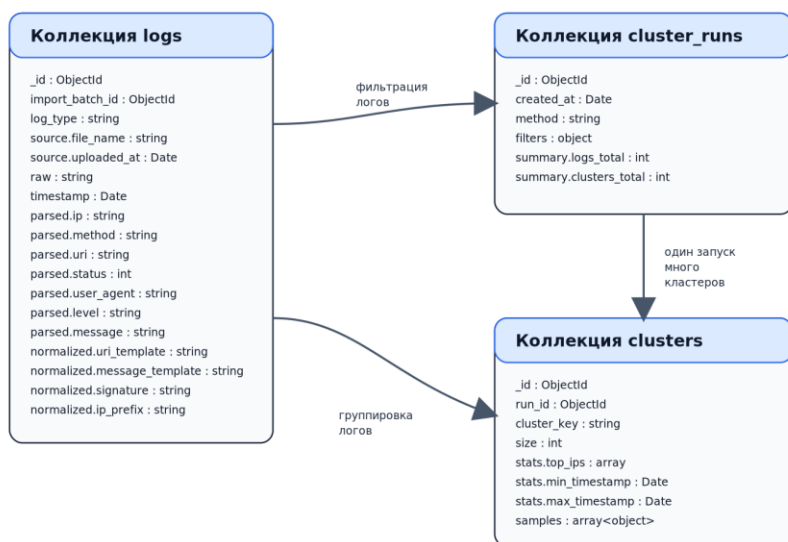


Рисунок 2 - Нереляционная модель данных MongoDB

Коллекция	Поле	Тип	Назначение
logs	_id	ObjectId	Идентификатор документа.
logs	import_batch_id	ObjectId	Идентификатор пакета импорта.
logs	log_type	string	Тип строки: access или error.
logs	source.file_name	string	Имя загруженного

			файла.
logs	source.uploaded_at	Date	Время загрузки.
logs	raw	string	Исходная строка лога.
logs	timestamp	Date	Дата и время события.
logs	parsed.ip	string	IP клиента для access-лога.
logs	parsed.method	string	HTTP-метод.
logs	parsed.uri	string	Запрошенный URI.
logs	parsed.status	int	HTTP-статус.
logs	parsed.user_agent	string	User-Agent клиента.
logs	parsed.level	string	Уровень error-лога.
logs	parsed.message	string	Текст error-сообщения.
logs	normalized.uri_template	string	URI с заменой динамических id на шаблон.
logs	normalized.message_template	string	Шаблон сообщения ошибки.
logs	normalized.ip_prefix	string	IP-префикс для группировки.
cluster_runs	method	string	Выбранный метод кластеризации.
cluster_runs	filters	object	Фильтры, примененные к logs.
cluster_runs	summary	object	Итоги запуска: logs_total, clusters_total.
clusters	run_id	string/ObjectId	Идентификатор запуска.
clusters	cluster_key	string	Ключ группировки.
clusters	size	int	Количество логов в кластере.
clusters	samples	array<object>	Примеры логов кластера.
clusters	stats	object	Счетчики и временные границы кластера.

Коллекция logs является центральной. Она хранит raw-строку, timestamp, поля. Коллекция cluster_runs хранит метаданные запусков: метод, фильтры, время создания и общую информацию. Коллекция clusters хранит рассчитанные группы: ключ кластера, размер и статистику.

Для оценки объема принимается средний размер одного документа logs 640 байт, одного документа cluster_runs 512 байт, одного документа clusters 336 байт. На каждые 1000 логов предполагается один запуск кластеризации, а один кластер в среднем соответствует 24 логам.

$$V_{\text{nosql}}(n) = 640n + 0.512n + 14n = 654.512n \text{ байт}$$

$$D_{\text{clean}}(n) = 300n \text{ байт}$$

$$R_{\text{nosql}}(n) = V_{\text{nosql}}(n) / D_{\text{clean}}(n) = 654.512n / 300n = 2.18$$

$$V_{\text{nosql}}(n) = O(n)$$

Запросы к нереляционной модели

Получение логов с многокритериальной фильтрацией:

```
db.logs.find(
  {
    log_type: "access",
    timestamp: {
      $gte: ISODate("2026-03-01T00:00:00Z"),
      $lt: ISODate("2026-03-08T00:00:00Z")
    },
    "parsed.status": { $in: [200, 404] },
    "parsed.uri": { $regex: "user", $options: "i" }
  }
)
```

Построение кластеров средствами MongoDB:

```
db.logs.aggregate([
  {
    $match: {
      log_type: "access",
      timestamp: {
        $gte: ISODate("2026-03-01T00:00:00Z"),
        $lt: ISODate("2026-03-08T00:00:00Z")
      }
    }
  },
  {
    $clusterOut: "clusters"
  }
])
```

```

    $group: {
      _id: "$normalized.signature",
      size: { $sum: 1 },
      sample_raw: { $first: "$raw" },
      top_ip: { $first: "$parsed.ip" }
    }
  },
  {
    $project: {
      _id: 0,
      cluster_key: "$_id",
      size: 1,
      samples: [{ raw: "$sample_raw" }],
      stats: { top_ip: "$top_ip" }
    }
  }
]

```

Для основных сценариев использования число запросов к БД можно оценить так:

просмотр и фильтрация логов - 1 запрос к logs, задействована 1 коллекция;

просмотр деталей выбранного лога - 1 запрос к logs, задействована 1 коллекция;

запуск кластеризации - 3 операции (aggregate по logs, insert в cluster_runs, сохранение результатов в clusters), задействованы 3 коллекции;

просмотр кластеров запуска - 1 запрос к clusters по run_id, задействована 1 коллекция;

просмотр логов кластера - 1 запрос к clusters и 1 запрос к logs, задействованы 2 коллекции.

Таким образом, для типовых пользовательских сценариев число запросов в нереляционной модели не зависит от количества логов как от числа отдельных документов. При росте числа логов меняется только стоимость обработки внутри одного запроса, но не число вызовов к БД. Для

фильтрации логов и просмотра деталей число запросов постоянно и равно 1, для сценариев кластеризации и показа логов кластера постоянно и равно 2-3.

3.2 Реляционная модель

SQL-аналог построен как схема из таблиц `import_batches`, `logs`, `cluster_runs`, `clusters` и `cluster_logs`. Основная проблема такой модели - необходимость хранить разные типы логов в одной широкой таблице.

Реляционная модель данных

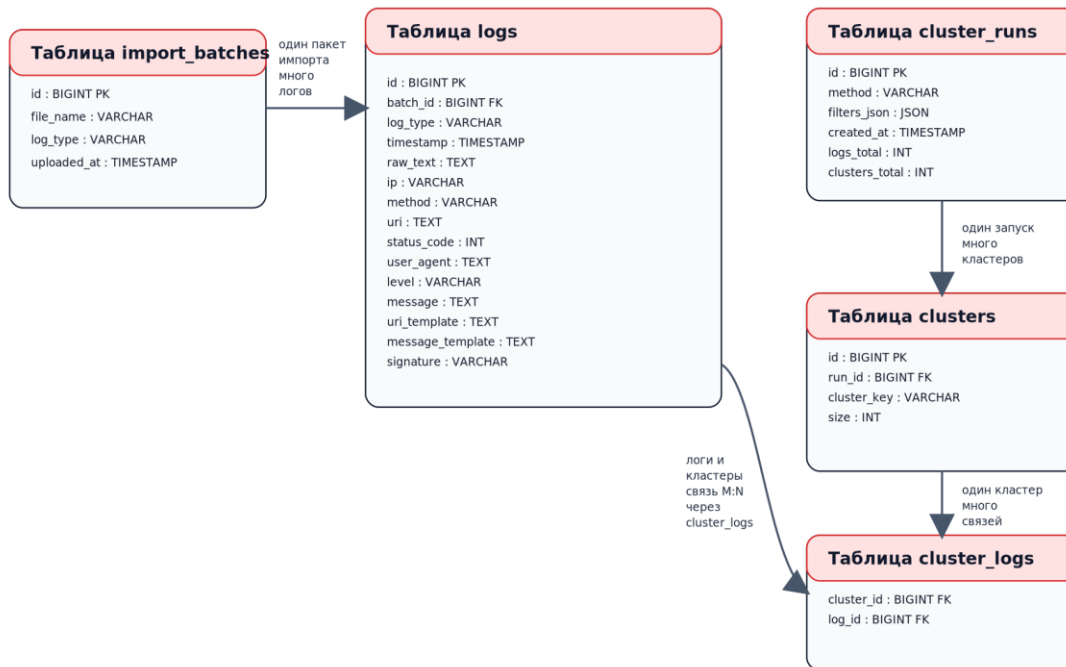


Рисунок 3 - Реляционная модель данных

Таблица	Поле	Тип	Назначение
import_batches	id	BIGINT PK	Пакет импорта.
import_batches	file_name	VARCHAR	Имя файла.
import_batches	uploaded_at	TIMESTAMP	Дата импорта.
logs	id	BIGINT PK	Запись лога.
logs	batch_id	BIGINT FK	Ссылка на import_batches.
logs	log_type	VARCHAR	access или error.
logs	timestamp	TIMESTAMP	Время события.
logs	raw_text	TEXT	Исходная строка.

logs	ip	VARCHAR	IP клиента.
logs	method	VARCHAR	HTTP-метод.
logs	uri	TEXT	URI запроса.
logs	status_code	INT	HTTP-статус.
logs	user_agent	TEXT	User-Agent.
logs	level	VARCHAR	Уровень ошибки.
logs	message	TEXT	Текст ошибки.
logs	uri_template	TEXT	Шаблон URI.
logs	message_template	TEXT	Шаблон сообщения.
cluster_runs	id	BIGINT PK	Запуск кластеризации.
cluster_runs	filters_json	JSON	Фильтры запуска.
clusters	id	BIGINT PK	Кластер.
clusters	run_id	BIGINT FK	Ссылка на запуск.
clusters	cluster_key	TEXT	Ключ кластера.
cluster_logs	cluster_id	BIGINT FK	Ссылка на clusters.
cluster_logs	log_id	BIGINT FK	Ссылка на logs.

В выбранном SQL-варианте таблица logs содержит поля как для access, так и для error записей. Поэтому часть колонок будет NULL. Для связи кластеров и логов нужна таблица cluster_logs, потому что один и тот же лог теоретически может попадать в разные кластеры разных запусков.

$$V_{sql}(n) = 592n + 24n + 2.67n + 0.128n = 618.798n \text{ байт}$$

$$D_{clean}(n) = 300n \text{ байт}$$

$$R_{sql}(n) = V_{sql}(n) / D_{clean}(n) = 618.798n / 300n = 2.06$$

$$V_{sql}(n) = O(n)$$

Запросы к реляционной модели

Получение логов с фильтрацией:

```
SELECT id, timestamp, ip, method, uri, status_code
FROM logs
WHERE log_type = 'access'
      AND timestamp >= '2026-03-01 00:00:00'
      AND timestamp < '2026-03-08 00:00:00'
      AND status_code IN (200, 404)
      AND LOWER(uri) LIKE '%user%';
```

Получение логов с фильтрацией:

```
SELECT l.id, l.timestamp, l.raw_text
```



```
FROM cluster_logs cl
JOIN logs l ON l.id = cl.log_id
WHERE cl.cluster_id = 101
ORDER BY l.timestamp;
```

Для основных сценариев использования число запросов к БД можно оценить так:

- просмотр и фильтрация логов - 1 запрос к logs, задействована 1 таблица;
- просмотр деталей выбранного лога - 1 запрос к logs, задействована 1 таблица;
- запуск кластеризации - 1 агрегирующий запрос по logs, 1 вставка в cluster_runs, серия вставок в clusters, серия вставок в cluster_logs; задействованы 4 таблицы;
- просмотр кластеров запуска - 1 запрос к clusters, задействована 1 таблица;
- просмотр логов кластера - 1 запрос с JOIN таблиц cluster_logs и logs, задействованы 2 таблицы.

Число пользовательских запросов здесь также остается постоянным, но сами сценарии кластеризации требуют большего числа записей в БД и большего числа задействованных таблиц.

3.3 Сравнение моделей

Для сравнения моделей введем обозначения:

n - количество сохраненных логов;

g - количество запусков кластеризации;

s - количество сохраненных кластеров;

k - количество логов, попавших в выбранный фильтр;

r - размер страницы при постраничном выводе;

m - количество кластеров в выбранном запуске.

Сравнение удельного объема информации

В нереляционной модели MongoDB основные данные хранятся в коллекциях logs, cluster_runs и clusters. Один лог хранится как один документ, содержащий исходную строку raw, распарсенные поля и подготовленные поля.

Средний размер одного документа logs равен 640 байт. Средний размер одного документа cluster_runs равен 512 байт. Средний размер одного документа clusters равен 336 байт.

При допущении, что на 1000 логов приходится один запуск кластеризации, а один кластер в среднем соответствует 24 логам, получаем:

$$r = n / 1000$$

$$c = n / 24$$

$$V_{\text{nosql}}(n) = 640n + 512r + 336c$$

$$V_{\text{nosql}}(n) = 640n + 512(n / 1000) + 336(n / 24)$$

$$V_{\text{nosql}}(n) = 640n + 0.512n + 14n = 654.512n \text{ байт}$$

В реляционной модели данные хранятся в таблицах import_batches, logs, cluster_runs, clusters и cluster_logs. Основной объем занимает таблица logs. Дополнительно требуется связующая таблица cluster_logs, так как связь между логам и кластерами реализуется явно.

Одна строка таблицы logs занимает 592 байта. На связь cluster_logs требуется 24 байта на один лог. Один кластер дает в среднем $64 / 24 = 2.67$ байта на один лог. Один запуск кластеризации дает $128 / 1000 = 0.128$ байта на один лог.

Тогда:

$$V_{\text{sql}}(n) = 592n + 24n + 2.67n + 0.128n$$

$$V_{\text{sql}}(n) = 618.798n \text{ байт}$$

Сравнение по объему:

$V_{\text{nosql}}(n) = 654.512n$ байт

Сценарий	MongoDB	SQL	Сложность MongoDB	Сложность SQL
Импорт логов	1 пакетная вставка в logs	вставка в import_batches и logs	$O(n)$	$O(n)$
Просмотр страницы логов	1 запрос к logs с limit/offset	1 запрос к logs с limit/offset	$O(\log n + p)$	$O(\log n + p)$
Фильтрация логов	1 запрос к logs по нескольким полям	1 запрос к logs по нескольким полям	$O(\log n + k)$	$O(\log n + k)$
Просмотр деталей лога	1 запрос к logs по _id	1 запрос к logs по id	$O(1)$	$O(1)$
Запуск кластеризации	aggregation по logs и запись cluster_runs и запись clusters	GROUP BY по logs + запись cluster_runs + clusters + cluster_logs	$O(k)$	$O(k)$
Просмотр запусков кластеризации	1 запрос к cluster_runs	1 запрос к cluster_runs	$O(\log r + p)$	$O(\log r + p)$
Просмотр кластеров запуска	1 запрос к clusters по run_id	1 запрос к clusters по run_id	$O(m)$	$O(m)$
Просмотр логов кластера	1-2 запроса: clusters и logs по samples/cluster_key	JOIN cluster_logs и logs	$O(s)$	$O(s \log n)$ с JOIN
Статистика запуска	агрегация по clusters/logs	GROUP BY с JOIN	$O(m)$	$O(k) + \text{JOIN}$
Экспорт данных	3 запроса к logs, cluster_runs, clusters	5 запросов к import_batches, logs,	$O(n + r + c)$	$O(n + r + c)$

Сценарий	MongoDB	SQL	Сложность MongoDB	Сложность SQL
		cluster_runs, clusters, cluster_logs		

Для простого просмотра логов обе модели имеют близкую сложность, так как данные читаются из одной основной сущности. Разница появляется в сценариях анализа и просмотра результатов кластеризации.

В MongoDB один документ logs уже содержит исходную строку, распарсенные поля и подготовленные признаки для группировки. Поэтому фильтрация и построение cluster_key выполняются внутри aggregation pipeline без соединения нескольких сущностей.

По удельному объему хранения SQL-модель в выбранной оценке занимает немного меньше памяти:

$$V_{\text{sql}}(n) = 618.798n \text{ байт}$$

$$V_{\text{nosql}}(n) = 654.512n \text{ байт}$$

Однако выигрыш по объему не является существенным для данной задачи. Основные сценарии приложения связаны не только с хранением, но и с чтением, фильтрацией, группировкой и анализом полуструктурированных логов.

NoSQL-модель лучше подходит для выбранной предметной области по следующим причинам:

- 1) один документ logs хранит все данные, необходимые для отображения и анализа конкретного лога;
- 2) access и error логи можно хранить в одной коллекции без большого количества NULL-полей;
- 3) подготовленные признаки позволяют выполнять кластеризацию через aggregation pipeline на стороне MongoDB;

4) результаты запусков кластеризации сохраняются отдельными документами в `cluster_runs` и `clusters`;

5) для большинства сценариев требуется меньше соединений между сущностями, чем в SQL-модели.

Таким образом, реляционная модель является рабочей, но менее естественной для задачи анализа Apache-логов. Для данного приложения более подходящей является MongoDB-модель, так как она лучше соответствует полуструктурированной природе логов и упрощает реализацию фильтрации, кластеризации и статистики.

4 РАЗРАБОТАННОЕ ПРИЛОЖЕНИЕ

4.1 Архитектура и контейнеры

Контейнер	Технология	Назначение
nsql-apache-app	Python, FastAPI, Uvicorn	Веб-интерфейс и REST API.
nsql-apache-db	MongoDB 7.0	Основное хранилище данных.
nsql-apache-mongo-express	mongo-express	Веб-просмотр базы при разработке и демонстрации.
mongo_data	Docker volume	Сохранение данных MongoDB между перезапусками.

Приложение обращается к базе по внутреннему адресу `mongodb://db:27017`. Наружу опубликованы только веб-приложение и `mongo-express` на `localhost`.

4.2 Модули приложения

Модуль	Назначение
app/main.py	Создание FastAPI-приложения, подключение статических страниц и роутов.
app/routes/api.py	REST API для сценариев импорта, экспорта,

	просмотра, кластеризации и статистики.
app/services/import_service.py	Разбор файлов, автоопределение access/error, защита replace от невалидного файла.
app/services/clustering_service.py	Построение MongoDB aggregation pipeline для кластеризации.
app/services/export_service.py	Сериализация коллекций в JSON-дампы.
app/services/log_generator.py	Генерация тестовых логов с распределением дат по периоду.
app/repositories	Изоляция запросов MongoDB и подготовка фильтров.
app/parsers	Парсинг строк Apache access и error.
app/static	Минимальный веб-интерфейс приложения.
scripts/generate_logs.py	CLI-обертка для запуска генератора из Docker.

4.3 REST API

Метод	Путь	Назначение
POST	/api/import	Импорт Apache-логов или JSON-дампы.
GET	/api/export	Экспорт всех данных приложения.
GET	/api/logs	Получение страницы логов с фильтрами.
GET	/api/logs/{id}	Получение деталей выбранного лога.
GET	/api/logs/{id}/raw	Получение исходной строки.
POST	/api/cluster-runs	Запуск кластеризации.
GET	/api/cluster-runs	Список запусков.
GET	/api/cluster-runs/{id}/clusters	Кластеры выбранного запуска.
GET	/api/cluster-runs/{id}/stats	Статистика выбранного запуска.
GET	/api/health	Проверка доступности приложения.

4.4 Методы кластеризации

Метод	Тип логов	Ключ кластера	Назначение
rule_based	access и error	access: method + uri_template + status; error: level + message_template	Универсальный метод для смешанного набора.
access_endpoint_status	access	method + uri_template + status	Поиск частых endpoint'ов и кодов ответа.
access_endpoint_status_ip	access	method + uri_template + status	Выделение групп запросов по

		+ ip_prefix	подсетям/IP.
error_level_message	error	level + message_template	Группировка повторяющихся ошибок.

Все методы выполняются по одному принципу: MongoDB получает выборку, строит cluster_key и группирует записи. Это соответствует требованию перенести максимум логики анализа на СУБД.

4.5 Снимки экрана приложения

Ниже на рисунках представлены снимки экрана приложения.

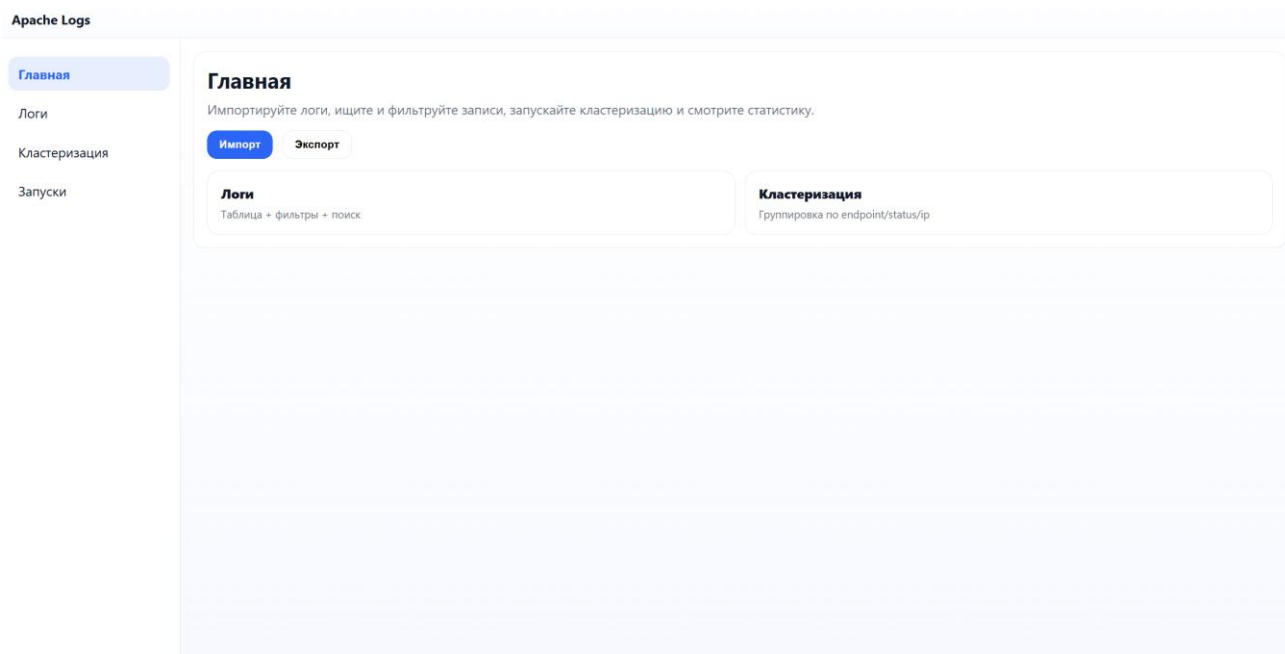


Рисунок 4 - Главная страница приложения

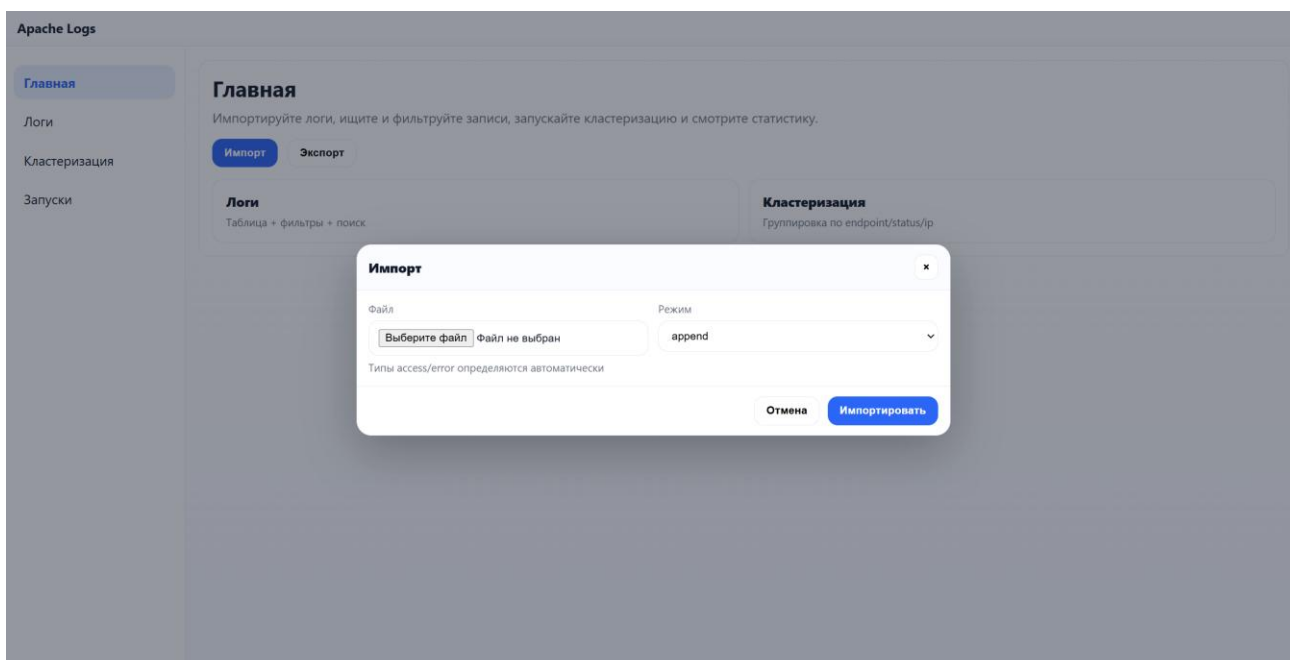


Рисунок 5 - Меню импорта логов

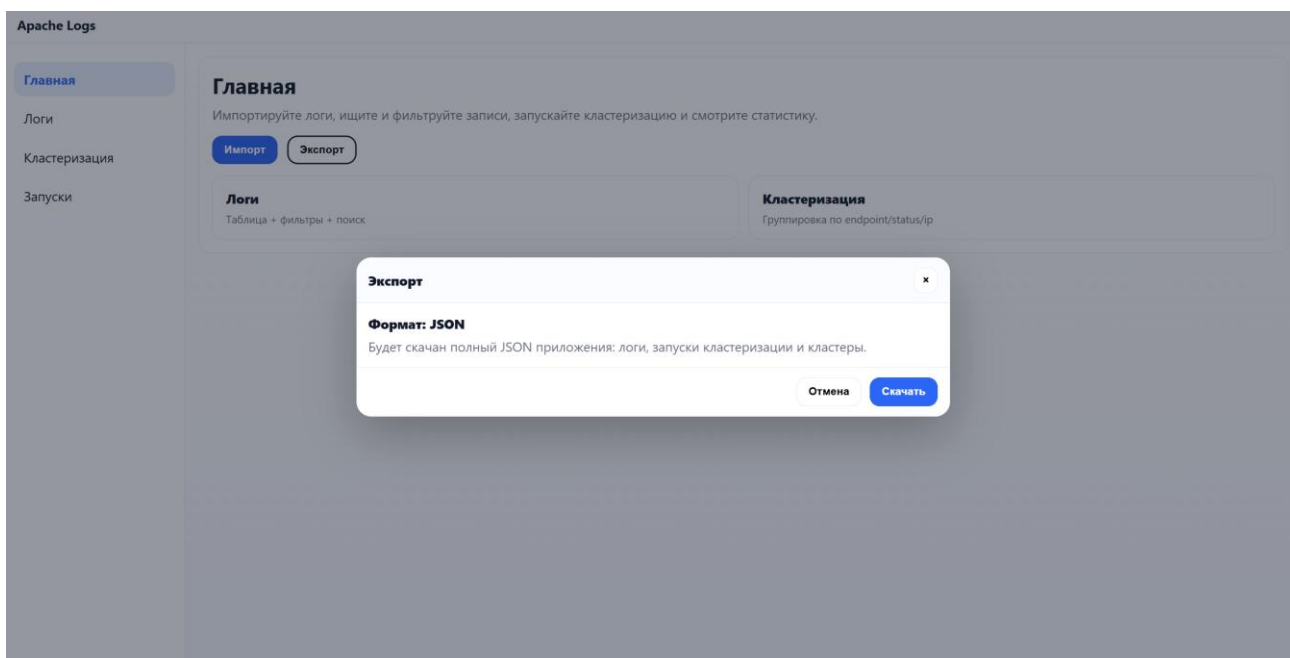


Рисунок 6 - Меню экспорта логов

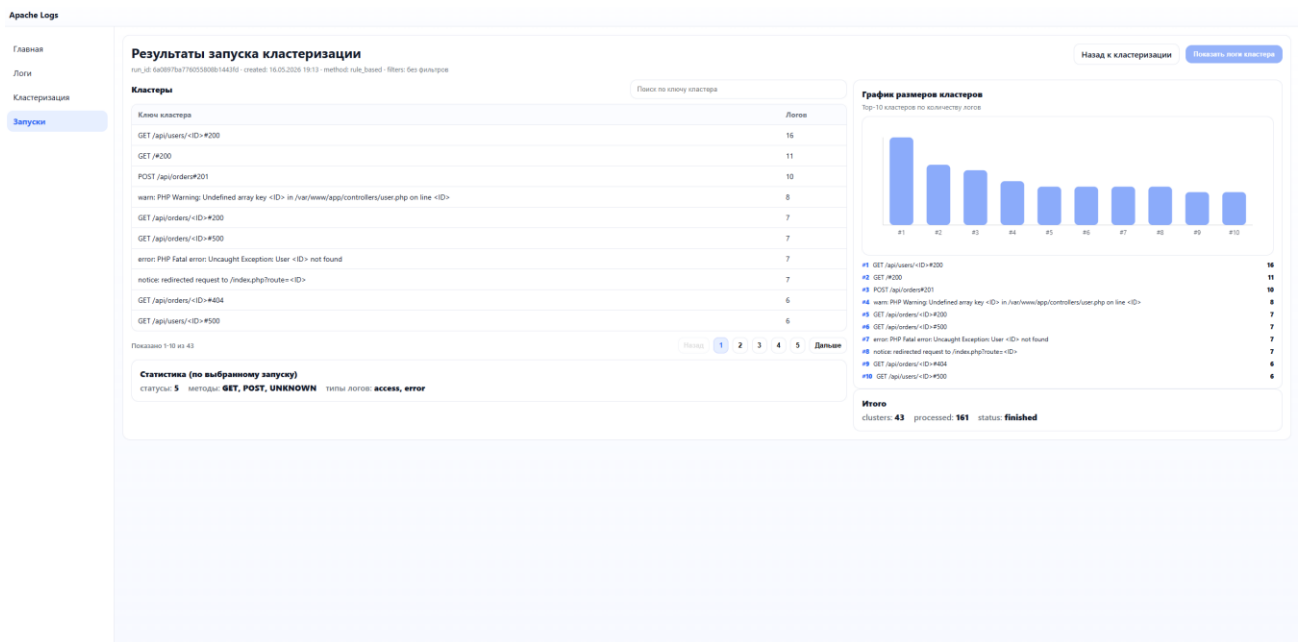


Рисунок 9 - Страница результатов запуска кластеризации

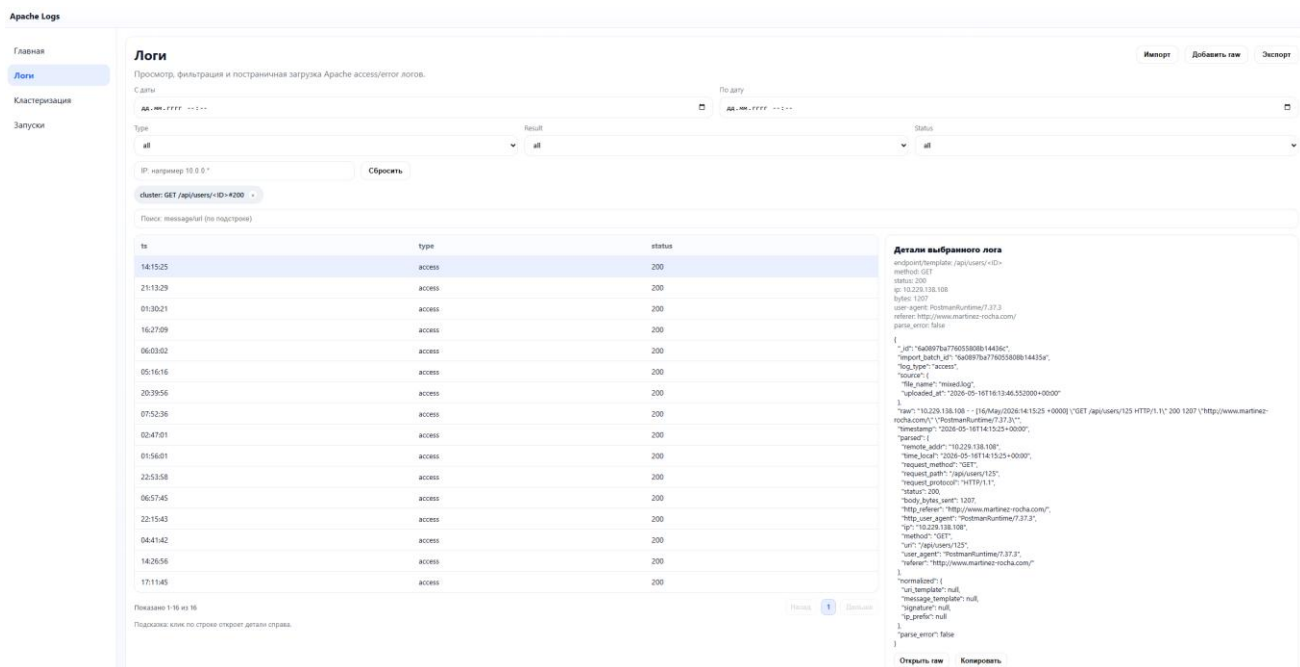


Рисунок 10 - Страница логов конкретного кластера

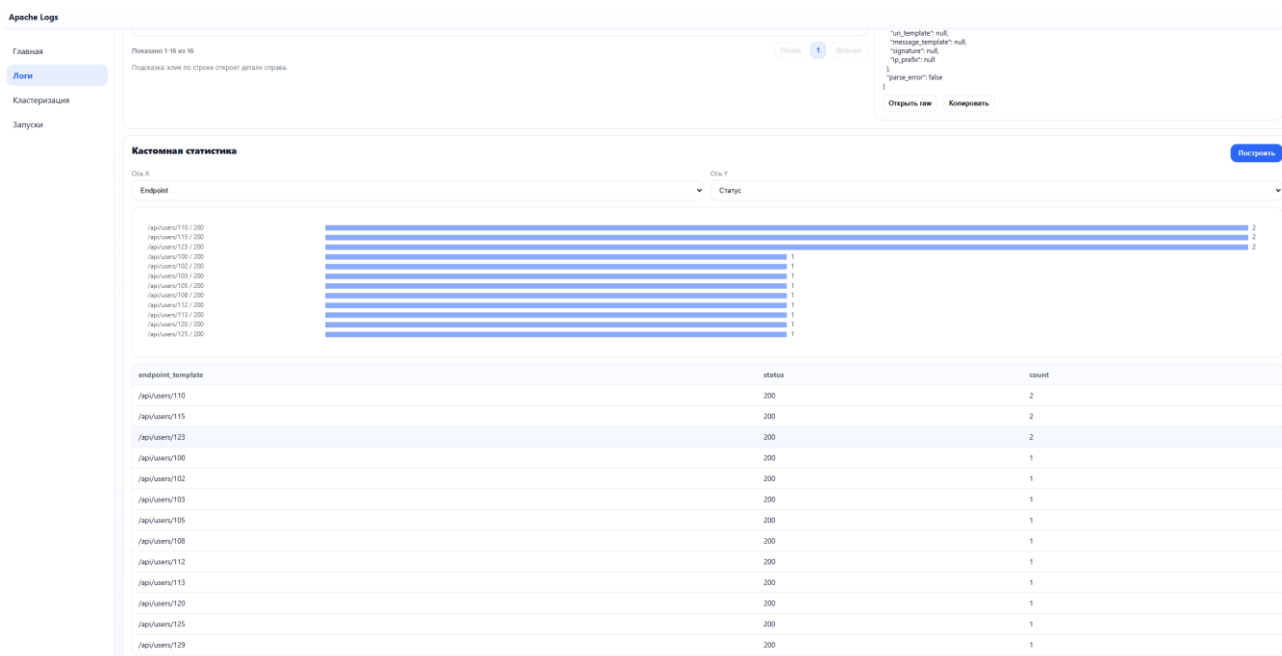


Рисунок 11 - Визуализация статистики конкретного кластера

5 ВЫВОД

В ходе работы разработано приложение для хранения, поиска, группировки и кластеризации логов Apache на MongoDB. Реализованы сценарии импорта, экспорта, просмотра, фильтрации, детализации, запуска кластеризации и просмотра статистики.

Недостатки текущего решения: ограниченный набор поддерживаемых форматов Apache, простые методы кластеризации, отсутствие полноценной авторизации и ограниченная визуализация статистики. В дальнейшем можно добавить полнотекстовый поиск, более сложные шаблоны нормализации, экспорт графиков и интеграцию с реальными источниками логов.

6 ПРИЛОЖЕНИЯ

6.1 Репозиторий проекта

Репозиторий проекта: <https://github.com/moevm/nsql1h26-apache>

6.2 Документация по сборке и развертыванию приложения

1. Установить git, Docker и Docker Compose.
2. Выполнить `git clone https://github.com/moevm/nsql1h26-apache.git`.
3. Перейти в каталог проекта: `cd nsql1h26-apache`.
4. Собрать контейнеры: `docker compose build --no-cache`.
5. Запустить приложение: `docker compose up -d`.
6. Открыть приложение: `http://localhost:8000`.
7. Сгенерировать тестовые логи командой `docker compose exec -T app python -m scripts.generate_logs --output-dir /app/generated_logs --access-count 300 --error-count 80 --days 10 --seed 42`.
8. Остановить приложение командой `docker compose down`. Для удаления данных выполнить `docker compose down -v`.

6.3 Инструкция пользователя

1. Открыть главную страницу приложения.
2. При необходимости сгенерировать тестовые файлы логов.
3. В разделе импорта выбрать файл логов и режим `append/replace`.
4. Открыть раздел логов, задать фильтры и перейти по страницам таблицы.
5. Выбрать лог, чтобы посмотреть его содержимое.
6. Открыть раздел кластеризации, выбрать метод и запустить анализ.
7. Открыть выбранный запуск, посмотреть кластеры, примеры и статистику.
8. Выполнить экспорт, чтобы получить JSON-дамп данных приложения.

7 СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. MongoDB Manual: <https://www.mongodb.com/docs/manual/>
2. MongoDB Aggregation Pipeline:
<https://www.mongodb.com/docs/manual/core/aggregation-pipeline/>
3. FastAPI Documentation: <https://fastapi.tiangolo.com/>
4. Apache HTTP Server Log Files:
<https://httpd.apache.org/docs/current/logs.html>
5. Docker Compose Documentation: <https://docs.docker.com/compose/>